



1. Quick Evaluation (Reality Check)

What you have is good, but here's the truth:

☑ Strong points

- Clear **core modules**: Items, Categories, Sales, Restock
- Good thinking on **filters, reports, dashboards**
- Nice attention to **UI/UX (older user, mobile-first)**



Gaps / Improvements

- No clear **database structure** (this is dangerous)
- Sales not fully defined (no cart, no transaction model)
- Restock is vague (needs history, not just levels)
- “Single user” assumption is limiting (design for scale)
- No mention of **stock deduction logic**



In short: concept is good, but backend logic needs structure.



2. System Architecture (Think Like ERP)

Break it into modules:

1. **Auth Module**
 2. **Inventory Module**
 3. **Sales Module**
 4. **Stock Management (Restock)**
 5. **Reports & Analytics**
-



3. Database Design (THIS IS THE HEART)

Let me give you a clean, normalized structure.



Users Table

```
users (  
  user_id INT PRIMARY KEY AUTO_INCREMENT,  
  username VARCHAR(50),  
  email VARCHAR(100),  
  password VARCHAR(255),  
  role ENUM('admin', 'user'),  
  created_at TIMESTAMP  
)
```



Categories Table

```
categories (  
  category_id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100),  
  created_at TIMESTAMP  
)
```



Items Table

```
items (  
  item_id INT PRIMARY KEY AUTO_INCREMENT,  
  category_id INT,  
  name VARCHAR(100),  
  description TEXT,  
  price DECIMAL(10,2),  
  stock_quantity INT DEFAULT 0,  
  restock_level INT DEFAULT 20,  
  created_at TIMESTAMP,  
  FOREIGN KEY (category_id) REFERENCES categories(category_id)  
)
```



Sales Table (Main Transaction)

```
sales (  
  sale_id INT PRIMARY KEY AUTO_INCREMENT,  
  total_amount DECIMAL(10,2),  
  sale_date TIMESTAMP,  
  user_id INT,  
  FOREIGN KEY (user_id) REFERENCES users(user_id)  
)
```



Sale Items Table (VERY IMPORTANT)

👉 This is where most students mess up.

```
sale_items (  
  sale_id INT,  
  item_id INT,  
  quantity INT,  
  price DECIMAL(10,2),  
  created_at TIMESTAMP  
)
```

```
sale_item_id INT PRIMARY KEY AUTO_INCREMENT,  
sale_id INT,  
item_id INT,  
quantity INT,  
price DECIMAL(10,2),  
subtotal DECIMAL(10,2),  
FOREIGN KEY (sale_id) REFERENCES sales(sale_id),  
FOREIGN KEY (item_id) REFERENCES items(item_id)  
)
```



Restocks Table

```
restocks (  
  restock_id INT PRIMARY KEY AUTO_INCREMENT,  
  item_id INT,  
  quantity INT,  
  restock_date TIMESTAMP,  
  user_id INT,  
  FOREIGN KEY (item_id) REFERENCES items(item_id),  
  FOREIGN KEY (user_id) REFERENCES users(user_id)  
)
```



Optional: Activity Logs (VERY SMART FOR ERP)

```
logs (  
  log_id INT PRIMARY KEY AUTO_INCREMENT,  
  action TEXT,  
  user_id INT,  
  created_at TIMESTAMP  
)
```



4. Key System Logic (Don't Miss This)

◆ When recording a sale:

- Insert into `sales`
- Insert multiple rows into `sale_items`
- Deduct stock:

```
stock_quantity = stock_quantity - quantity
```

◆ When restocking:

- Insert into `restocks`
- Increase stock:

```
stock_quantity = stock_quantity + quantity
```

◆ **Low stock alert:**

```
WHERE stock_quantity <= restock_level
```

5. Now Your Technical Prompt (Clean & Powerful)

You can use this for development or even AI coding assistance 🙌

TECHNICAL PROMPT

Build a modular web-based ERP system for a hardware inventory and sales management system using:

- Frontend: HTML5, CSS, Bootstrap, JavaScript, jQuery
- Backend: PHP (procedural or MVC) with AJAX
- Database: MySQL

Core Requirements:

1. **Authentication System**
 - User registration, login, logout
 - Password hashing
 - Session management with auto logout on inactivity
2. **Inventory Management**
 - CRUD operations for categories
 - CRUD operations for items under categories
 - Each item must include:
 - Name, category_id, price, stock_quantity, restock_level
3. **Sales Module**
 - Ability to create a sale transaction
 - Select multiple items with quantities (cart-like system)
 - Store sales in:
 - sales table (main transaction)
 - sale_items table (line items)
 - Automatically deduct stock after sale
4. **Restock Module**
 - Record restocking transactions
 - Increase stock quantities
 - Maintain restock history
5. **Dashboard & Analytics**
 - Display:

- Daily, weekly, monthly sales
 - Total revenue
 - Low stock alerts
 - Use charts (line & bar graphs)
 - 6. **Reports**
 - Filter by:
 - Date range
 - Item
 - Category
 - Generate printable reports
 - 7. **UI/UX Requirements**
 - Sidebar navigation with collapsible menus
 - Mobile-first responsive design
 - Large buttons for usability (age 40+ users)
 - Confirmation dialogs for destructive actions
 - Undo support where possible
 - 8. **System Behavior**
 - All data views must support filtering
 - Use AJAX for dynamic updates (no full page reloads)
 - Maintain clean separation of concerns (PHP, JS, CSS)
 - 9. **Database Integrity**
 - Use foreign keys
 - Normalize tables (avoid redundancy)
 - Ensure transactional consistency for sales
-



Final Advice (Real Talk)

If you build this:

- Don't rush UI first → **start with database + logic**
- Test flows like:
 - "Sell item with low stock"
 - "Restock then sell"
- Keep queries simple (since you're on InfinityFree)